



Hochschule für Angewandte Wissenschaften Hamburg
Hamburg University of Applied Sciences

Informatik 1

BA Medieninformatik

Prof. Dr.-Ing. Sabine Schumann

Fakultät Informatik und Digitale Gesellschaft, Fachgruppe Medieninformatik

Dateisysteme & Datenbanksysteme

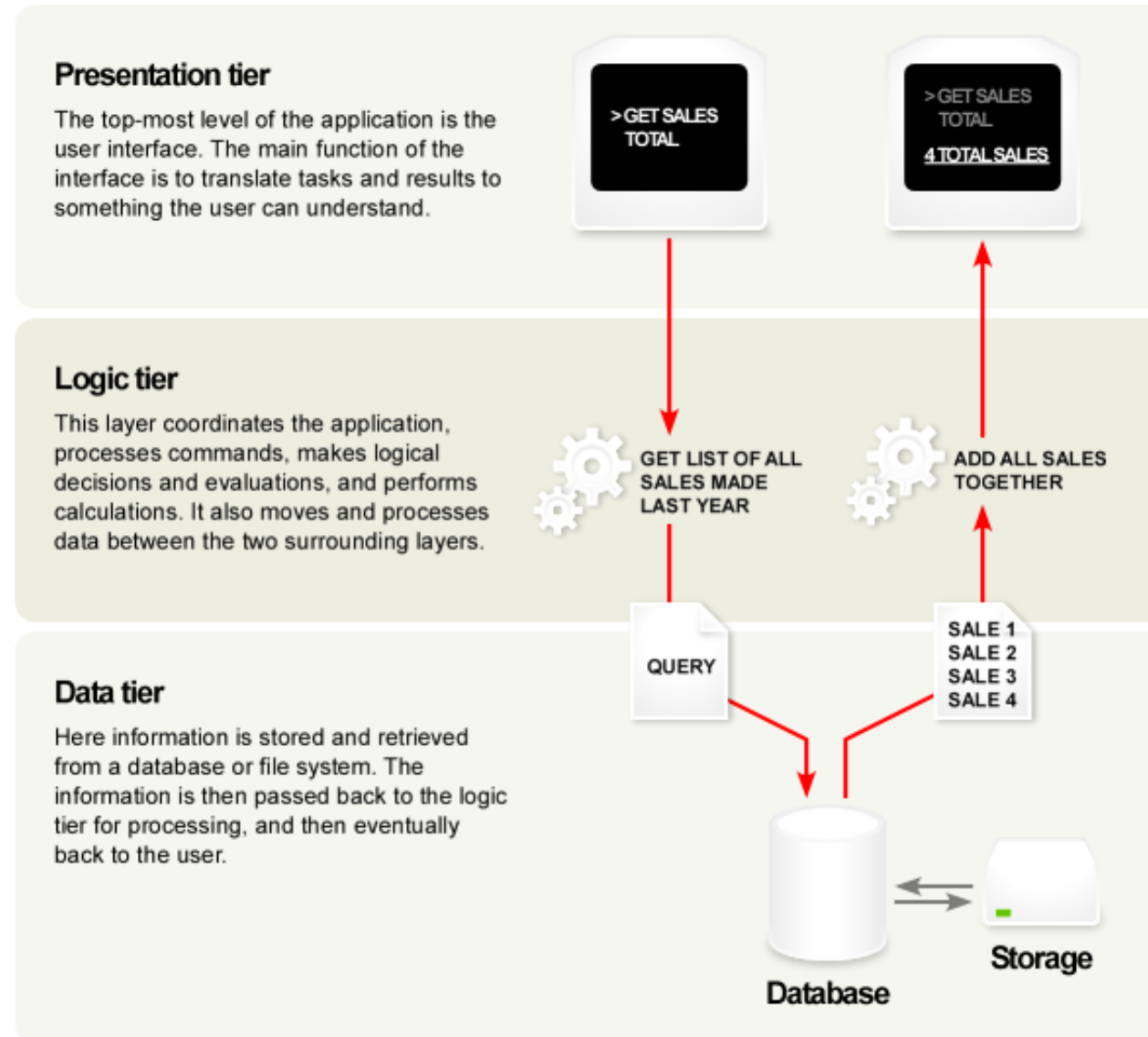
Was erwartet Sie in diesem Kapitel?

3 Ebenen Architektur

Dateisysteme

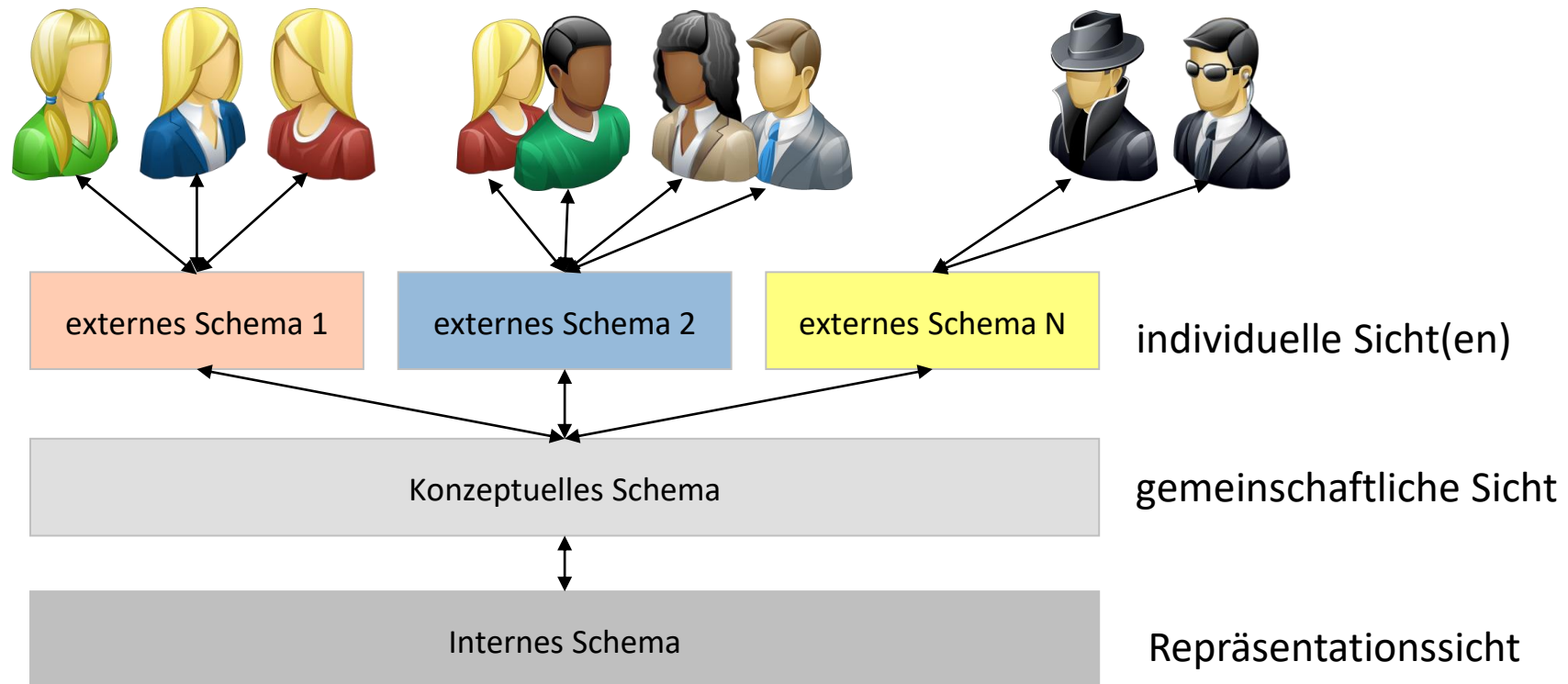
Datenbanken

3 Ebenen Architektur (1)



3 Ebenen Architektur (2)

- zur Unterstützung der Datenunabhängigkeit,
- 1975 von ANSI / SPARC entwickelt
ANSI: **A**merican **N**ational **S**tandards **I**nstitute
SPARC: **S**tandards **P**lanning and **R**equirements **C**ommittee



Dateisysteme

Datei – Dateisystem – Dateiverwaltung

Datei

- Sammlung von Daten, die auf einem Permanentpeicher gehalten wird
- Daten in einer Datei können in unterschiedlicher Form organisiert sein

Dateisystem

- verantwortlich dafür, die Daten auf einem Speichermedium in geeigneter Form zugänglich zu machen
- Nutzer*in muss sich **nicht** um die Details der internen Datenorganisation kümmern
- bietet Schutzmechanismus, so dass Dateien nur von Nutzer*innen gelesen bzw. manipuliert werden dürfen, die eine Berechtigung dafür besitzen
- Dateisysteme werden vom Betriebssystem zur Verfügung gestellt

Dateisystem

Ein Dateisystem (englisch File System) ist eine Menge von abstrakten Datentypen (ADT).

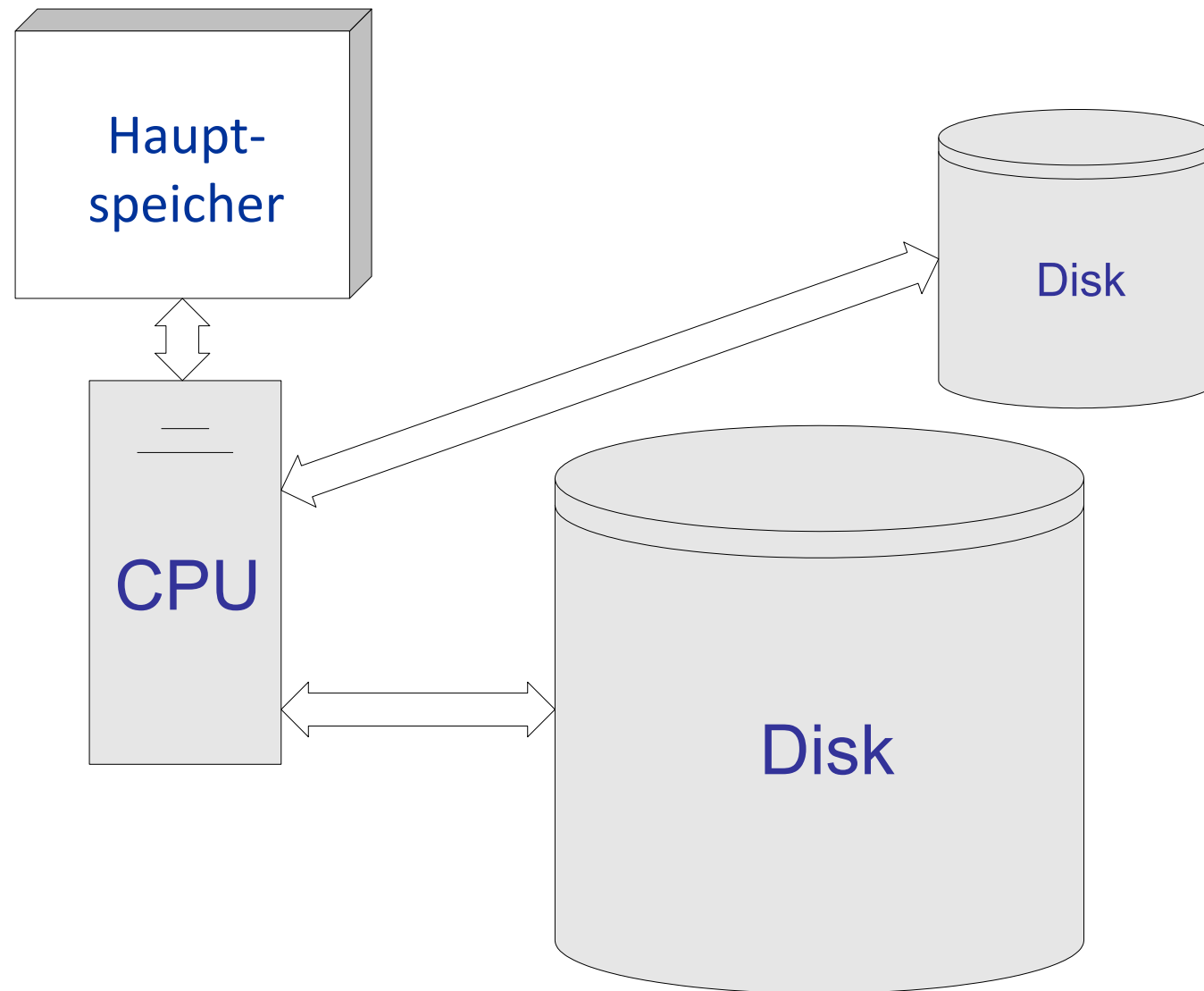
Diese ADT sind für die

- Speicherung,
- hierarchische Organisation,
- Manipulation,
- Navigation,
- Zugriff und
- Abruf

von Daten implementiert.

 Verwaltungsinformationen werden getrennt von den eigentlichen Daten gespeichert!

Dateisystem



Arten von Dateien

- „normale Dateien“
(Textdateien, ausführbare Programme, Bilddateien, ...)
- Verzeichnisse (Directories)
- Verweise (Links) auf andere Dateien etc.

i.a. werden Dateien in Form eines **byte-stream** gespeichert

- einfachste und häufigste Form
- lineare Folge von Daten (Bytes)

Dateisystem Aufbau

- ein Dateisystem legt eine Blockgröße fest, d.h. alle **Daten**blöcke in dem Dateisystem sind gleich groß
- im Sinne der Dateiverwaltung ist die Datenblockgröße die kleinste zu adressierende Einheit
- eine Datei wird durch einen inode beschrieben
- im inode stehen Informationen zur Datei (Eigentümer, Größe, ...)
- + Verweise (Zeiger) auf die Datenblöcke, wo die „richtigen“ Daten aufgehoben sind

Das inode Prinzip am Beispiel ext2-Dateisystem

- ext ... extended File System
- ext2 wurde 1993 von Rémy Card für Linux entwickelt
- die Verwaltungsdaten einer Datei (Rechte, Größe, ...) werden in sogenannten *Inodes* gehalten
- der Name der Datei steht **nicht** im inode
(der inode und der Dateiname befinden sich in der Verzeichnisdatei)
- die Größe des inodes ist unabhängig von der Blockgröße der Datenblöcke und beträgt bei ext2 konstant 128 Byte

ext2-Dateisystem (1)

wenn man die Daten direkt im inode speichern würde,
hätte man zu wenig Platz (nur noch 60 Byte) → unpraktikabel

12 direkte Zeiger (Adressen) auf Datenblöcke, d.h. diese werden direkt adressiert

- bei einer Datenblockgröße von **bspw. 1 KByte** (=1024 Byte) lassen sich maximal $12 * 1024$ Bytes (= 12 KByte) speichern
- Dateien, die relativ klein sind, können schnell gelesen werden

reicht der Speicherplatz dieser 12 Datenblöcke nicht aus ...

ext2-Dateisystem (2)

reicht der Speicherplatz dieser 12 Datenblöcke nicht aus ...

einfacher indirekter Zeiger

- der nächste Eintrag im inode referenziert einen Datenblock, der keine „echten“ Daten enthält, stattdessen Zeiger (Adressen) auf die eigentlichen Datenblöcke
- er verweist somit auf einen Block, der (falls ein Zeiger 4 Byte belegt) maximal $\frac{1024}{4} = 256$ Zeiger (Adressen) aufnehmen kann
- demzufolge kann man über diesen einfachen indirekten Zeiger $256 * 1024$ Bytes (256 KByte) adressieren

reicht der Speicherplatz der 12 Datenblöcke und auch der des einfach indirekten Zeigers nicht aus ...

ext2-Dateisystem (3)

reicht der Speicherplatz der 12 Datenblöcke und auch der des einfach indirekten Zeigers nicht aus ...

zweifach indirekter Zeiger

- der nächste Eintrag im inode referenziert einen Datenblock, der keine „echten“ Daten enthält, stattdessen Zeiger (Adressen) auf Datenblöcke, die wiederum Zeiger (Adressen) auf die eigentlichen Datenblöcke enthalten
- über diesen zweifach indirekten Zeiger können
256 * 256 * 1024 Bytes (65 536 KByte) adressiert werden

reicht der Speicherplatz der 12 Datenblöcke und der des einfach indirekten Zeigers und der des zweifach indirekten Zeigers nicht aus ...

ext2-Dateisystem (4)

reicht der Speicherplatz der 12 Datenblöcke und der des einfach indirekten Zeigers und der des zweifach indirekten Zeigers nicht aus ...

dreifach indirekter Zeiger

- der letzte Eintrag im inode ist der dreifach indizierte Zeiger
- über diesen dreifach indirekten Zeiger können
(bei unserer beispielhaft gewählten Datenblockgröße von 1 KByte)
 $256 * 256 * 256 * 1024$ Bytes (16 GByte) adressiert werden

Datenblockgrößen ext2

- inode hat bei ext2 konstant 128 Byte
- Datenblockgröße wird dagegen beim Einrichten des Dateisystems festgelegt
- zulässig sind 1, 2 und 4 KByte
- Ermittlung der maximalen Dateigröße:
die vier Werte addieren

Blockgröße (KByte)	theoretisch maximale Dateigröße (GByte)
1	16 $12+256+256*256+256*256*256$
2	256 $12*2+512*2+512*512*2+512*512*512*2$
4	4100 $12*4+1024*4+1024*1024*4+1024*1024*1024*4$

Der Datenblock ist die kleinste zu adressierende Einheit!

Welche KByte-Größe ist die beste Wahl?

Für welche Datenblockgröße würden Sie sich entscheiden?

Diskutieren Sie miteinander und finden Sie Argumente für Ihre gewählte Datenblockgröße und Gegenargumente, die gegen die restlichen Datenblockgrößen Ihrer Ansicht nach sprechen.



10 Minuten

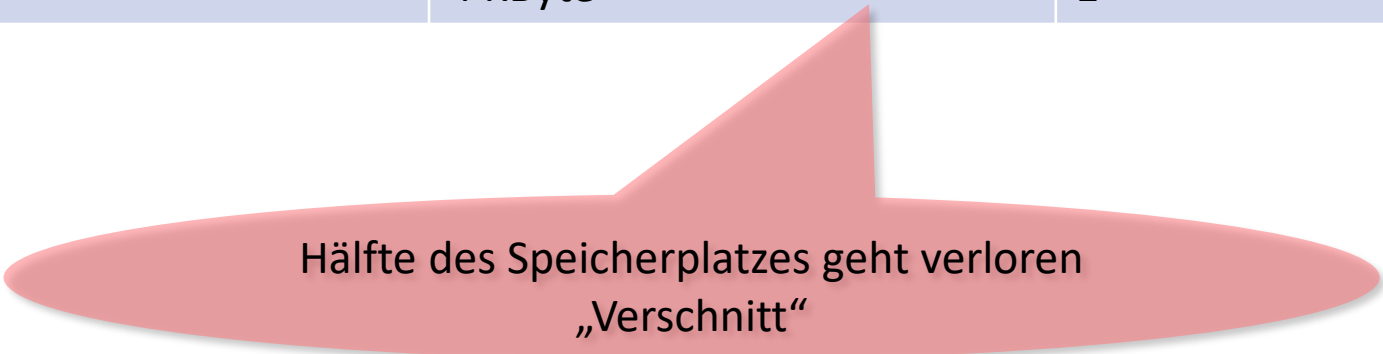
Theorie und Praxis

der physische Speicherplatzverbrauch einer Datei ist im Mittel größer als ihr tatsächlicher Bedarf

Beispiel:

1200 Byte große Datei

Blockgröße (KByte)	benötigter Speicherplatz	Anzahl Datenblöcke
1	2 KByte	2
2	2 KByte	1
4	4 KByte	1



Hälfte des Speicherplatzes geht verloren
„Verschnitt“

Verschnitt

... ist deswegen 1 KByte Datenblockgröße die beste?

- ja, für überwiegend kleine Dateien
- bei großen Dateien beschleunigt die 4 KByte-Datenblockgröße den Lesezugriff erheblich, da sichergestellt ist, dass umfangreiche Teile der Datei hintereinander auf der Festplatte liegen und der Lese-/Schreibkopf sie sehr schnell finden kann
- große Blockgrößen minimieren den Effekt der Fragmentierung
- (Gegenmaßnahme:
die Festplatte ab und an defragmentieren ...)

eine neue Datei wird erzeugt

- finden eines freien inode
- reservieren einer Reihe möglichst zusammenhängender Datenblocknummern
- diese werden solange reserviert bis die Datei wirklich abgespeichert wurde
- danach werden nicht benutzte Datenblöcke wieder freigegeben

Lernziele Dateisysteme

... Organisationsstruktur erklären können

... Vorteile und Nachteile der unterschiedlichen Blockgrößen bewerten können

Datenbanksysteme

Entstehung von Datenbanksystemen

- Anwendungen schreiben ihre Daten als Dateien in das Dateisystem, bspw.
 - alle Kundendaten
 - alle Artikel
 - alle Bestellungen
 - alle Lagerbestände
 - ...
- andere Anwendungen wollen auf diese Daten zugreifen (lesend / schreibend)

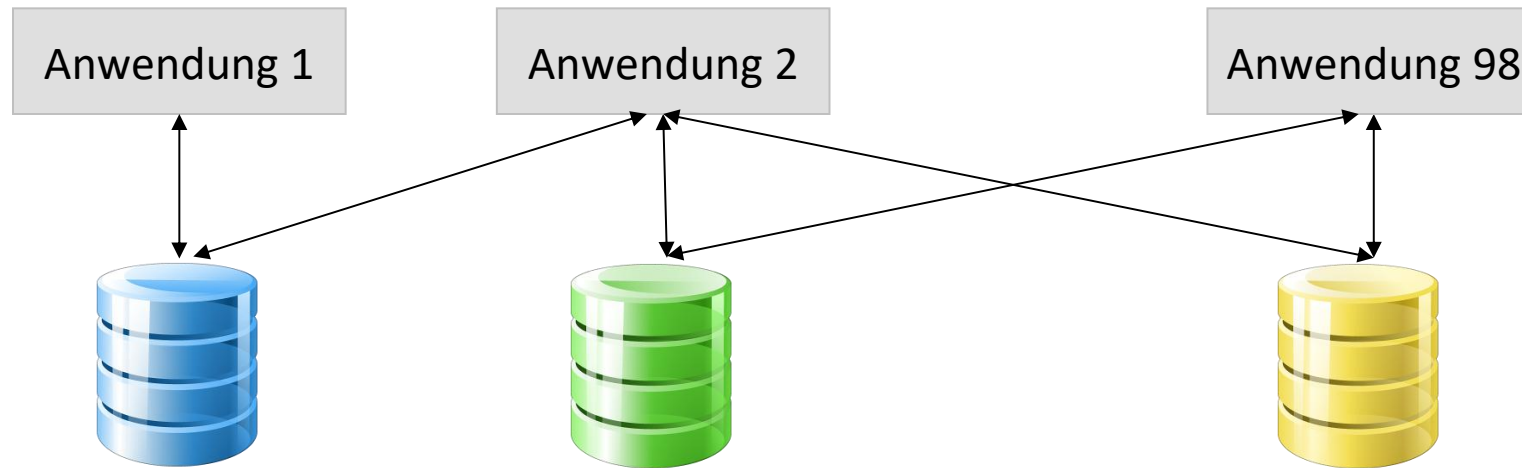
Speicherung von Daten in Dateisystemen

in der Anfangszeit der Datenverarbeitung ...

- Daten, die durch ein Programm erzeugt und verarbeitet werden, sind in anwendungseigenen Dateiformaten strukturiert und gespeichert
- Nachteil:
unterschiedliche Anwendungsprogramme können Daten nicht oder nur durch hohen Aufwand über die Programmierung von Schnittstellen austauschen
- die Dateiverwaltung führt meist zu Redundanz, Inkonsistenz und Datenabhängigkeit

Entstehung von Datenbanksystemen

Datenbanksysteme sind historisch aus **Dateisystemen** entstanden



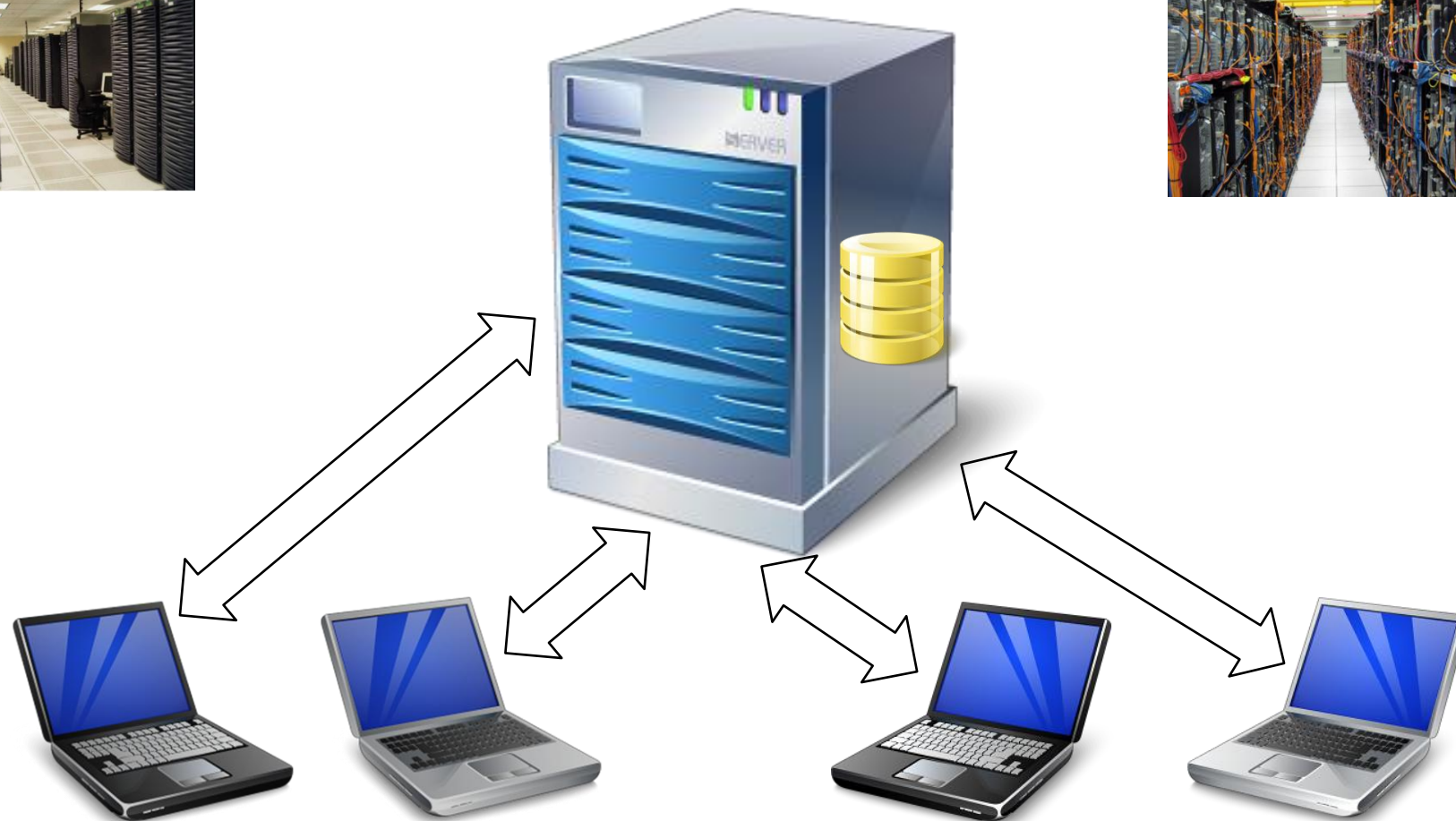
Die alleinige Verwendung von Dateisystemen funktionierte (und funktioniert) gut, solange die Anzahl der Anwendungen gering war/ist und die Anwendungen möglichst wenig von anderen Dateien nutzen.

Speicherung der Daten in Datenbanksystemen

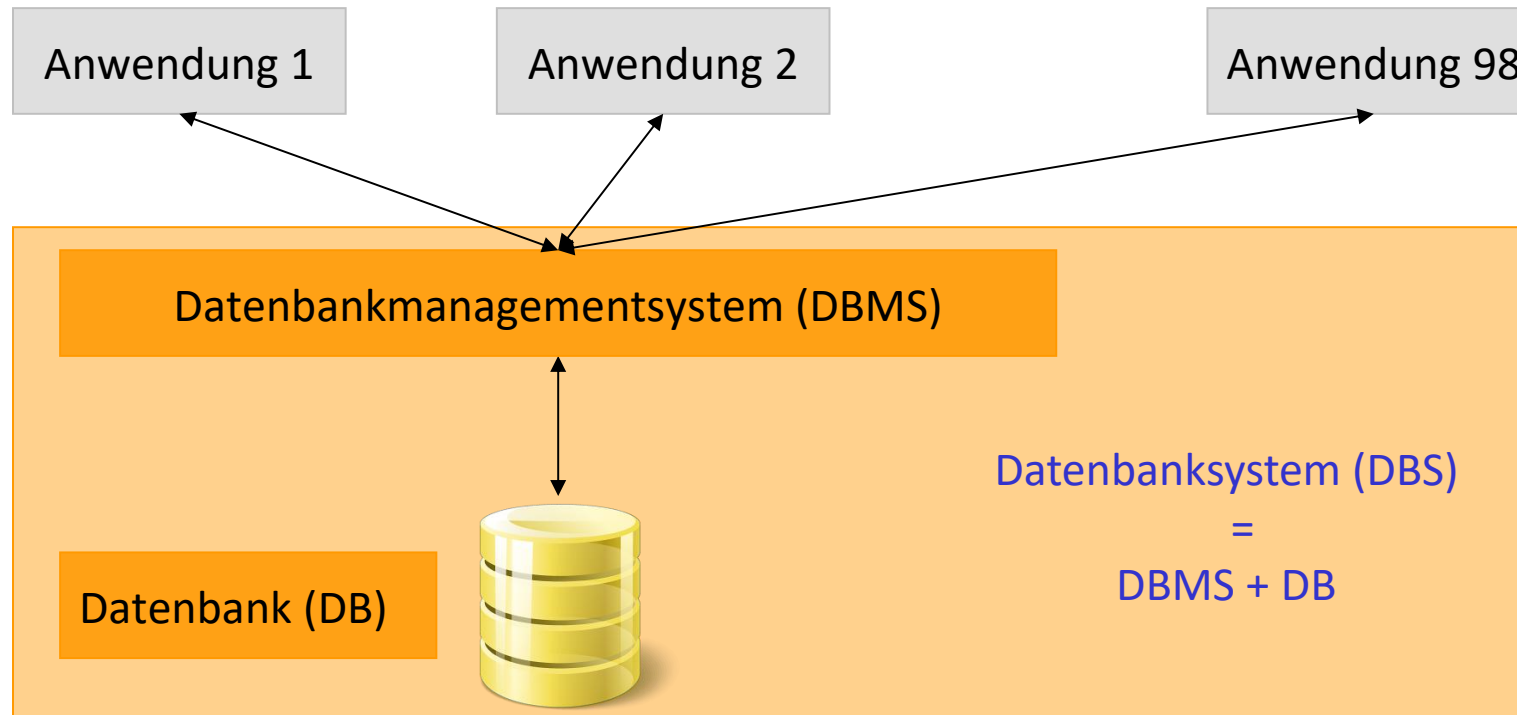
ein Datenbanksystem

- unterstützt die computergestützte Datenverarbeitung von Informationen, die durch eine Datenbankapplikation erzeugt und verarbeitet werden
- strukturiert die Informationen und speichert sie in einer Datenbank ab

Datenbanksystem – Client-Server



Datenbanksysteme



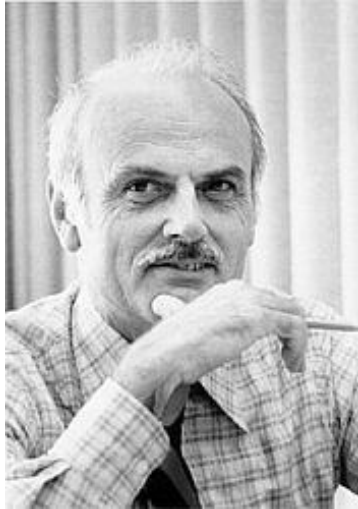
Vorteile Datenbanksysteme vs. Dateisystem

- geringer Erstellungs- und Verwaltungsaufwand
- sichere Realisierung des gleichzeitigen Zugriffs auf Daten durch mehrere Nutzer*innen / Programme
- Änderungen am Format der Daten ziehen keine Änderungen in den darauf zugreifenden Programmen nach sich
(*Datenunabhängigkeit*)
- Vermeidung von Redundanz der Daten
(es existiert genau ein Satz von Daten, nicht mehrere wie in Dateisystemen)
- Vermeidung von Inkonsistenzen
(... weil nur ein Datensatz der realen Welt existiert)
- Datensicherung und Zugriffskontrolle findet für die Datenbank insgesamt statt, nicht getrennt und verstreut für Einzeldateien

Aufgaben von Datenbanksystemen

1. einheitliche, redundanzfreie Datenverwaltung (Integration)
2. Operationen: Speichern, Ändern, Löschen, Suchen/Lesen
3. Zugriffe auf Datenbankbeschreibungen (Data Dictionary, Katalog)
4. Festlegung benutzerspezifischer Sichten auf Relationen
5. Integritätssicherung / Konsistenzüberwachung:
Korrektheit des Datenbankinhaltes
6. Verhinderung unauthorisierter Zugriffe (Datenschutz)
7. mehrere Operationen zu einer logischen Einheit zusammenfassen
(Transaktionskonzept)
8. Synchronisation im Mehrnutzerbetrieb
9. Möglichkeit zur Sicherung des Datenbestandes und Wiederherstellung nach Systemfehlern

Relationale Datenbanken



Edgar Frank „Ted“ Codd
(1923-2003)

Das relationale Daten(bank)modell wurde 1970 von Edgar Frank „Ted“ Codd (IBM) in seiner Dissertation vorgestellt.

(Dafür gab es den Turing-Award, die höchste Auszeichnung („Nobelpreis“) für Informatiker)

A Relational Model of Data for Large Shared Data Banks

E. F. Codd

IBM Research Laboratory, San Jose, California

Future users of large data banks must be protected from having to know how the data is organized in the machine (the internal representation). A prompting service which supplies such information is not a satisfactory solution. Activities of users at terminals and most application programs should remain unaffected when the internal representation of data is changed and even when some aspects of the external representation are changed. Changes in data representation will often be needed as a result of changes in query, update, and report traffic and natural growth in the types of stored information.

Existing noninferential, formatted data systems provide users with tree-structured files or slightly more general network models of the data. In Section 1, inadequacies of these models are discussed. A model based on n -ary relations, a normal form for data base relations, and the concept of a universal data sublanguage are introduced. In Section 2, certain operations on relations (other than logical inference) are discussed and applied to the problems of redundancy and consistency in the user's model.

KEY WORDS AND PHRASES: data bank, data base, data structure, data organization, hierarchies of data, networks of data, relations, derivability, redundancy, consistency, composition, join, retrieval language, predicate calculus, security, data integrity

CR CATEGORIES: 3.70, 3.73, 3.75, 4.30, 4.22, 4.29

1. Relational Model and Normal Form

1.1. INTRODUCTION

This paper is concerned with the application of elementary relation theory to systems which provide shared access to large banks of formatted data. Except for a paper by Childs [1], the principal application of relations to data systems has been to deductive question-answering systems. Levein and Maron [2] provide numerous references to work in this area.

In contrast, the problems treated here are those of *data independence*—the independence of application programs and terminal activities from growth in data types and changes in data representation—and certain kinds of *data inconsistency* which are expected to become troublesome even in nondeductive systems.

The relational view (or model) of data described in Section 1 appears to be superior in several respects to the graph or network model [3, 4] presently in vogue for non-inferential systems. It provides a means of describing data with its natural structure only—that is, without superimposing any additional structure for machine representation purposes. Accordingly, it provides a basis for a high level data language which will yield maximal independence between programs on the one hand and machine representation and organization of data on the other.

A further advantage of the relational view is that it forms a sound basis for treating derivability, redundancy, and consistency of relations—these are discussed in Section 2. The network model, on the other hand, has spawned a number of confusions, not the least of which is mistaking the derivation of connections for the derivation of relations (see remarks in Section 2 on the “connection trap”).

Finally, the relational view permits a clearer evaluation of the scope and logical limitations of present formatted data systems, and also the relative merits (from a logical standpoint) of competing representations of data within a single system. Examples of this clearer perspective are cited in various parts of this paper. Implementations of systems to support the relational model are not discussed.

1.2. DATA DEPENDENCIES IN PRESENT SYSTEMS

The provision of data description tables in recently developed information systems represents a major advance toward the goal of data independence [5, 6, 7]. Such tables facilitate changing certain characteristics of the data representation stored in a data bank. However, the variety of data representation characteristics which can be changed *without logically impairing some application programs* is still quite limited. Further, the model of data with which users interact is still cluttered with representational properties, particularly in regard to the representation of collections of data (as opposed to individual items). Three of the principal kinds of data dependencies which still need to be removed are: ordering dependence, indexing dependence, and access path dependence. In some systems these dependencies are not clearly separable from one another.

1.2.1. *Ordering Dependence.* Elements of data in a data bank may be stored in a variety of ways, some involving no concern for ordering, some permitting each element to participate in one ordering only, others permitting each element to participate in several orderings. Let us consider those existing systems which either require or permit data elements to be stored in at least one total ordering which is closely associated with the hardware-determined ordering of addresses. For example, the records of a file concerning parts might be stored in ascending order by part serial number. Such systems normally permit application programs to assume that the order of presentation of records from such a file is identical to (or is a subordering of) the

Relationes Daten(bank)modell

Person

PersonID	Nachname	Vorname	Alter	Stadt	Straße

Artikel

ArtikelNr	Artikelname	Preis

Bestellung

BestellNr	ArtikelNr	PersonID	Menge	Datum



physische Speicherung im Filesystem (1)

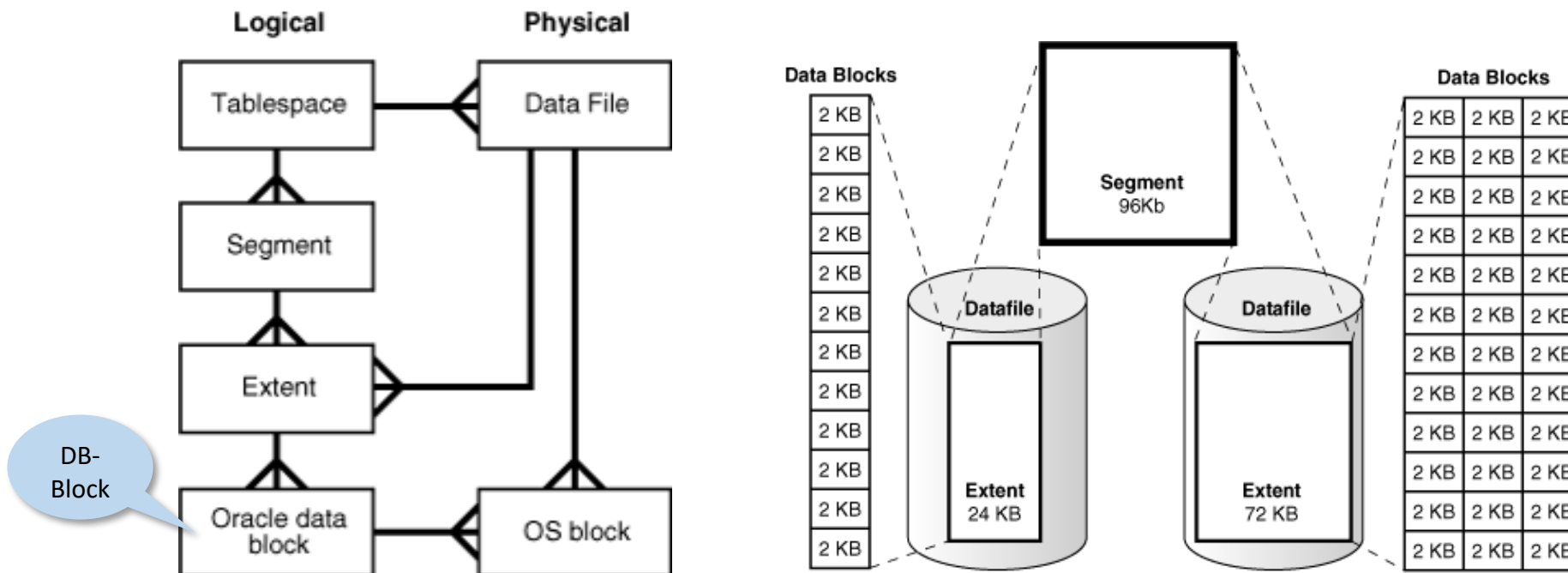
Beim Anlegen der Datenbank wird diese in **gleich große** Blöcke (**DB-Block**) eingeteilt.

DB-Block:

- kleinste logische Speichereinheit der Datenbank
- Größe abhängig von der physischen Blockgröße des Dateisystems (OS-Block)
 - DB-Block-Größe $\geq$ OS-Block-Größe
 - DB-Block-Größe Vielfaches der OS-Block-Größe
- DB-Blocks werden zu größeren „Einheiten“ sogenannte **Extents** logisch zusammengefasst
- Extents werden wiederum zu größeren „Einheiten“ sogenannten **Segments** logisch zusammengefasst
- Segmente werden logisch in **Tablespaces** organisiert

physische Speicherung im Filesystem

- Tablespaces werden **physisch** in einem oder mehreren Datenfiles gespeichert
- dabei muss ein Extent komplett in einem Datenfile gespeichert werden, Segmente können über mehrere Dateien verteilt sein



Bildquellen: http://download.oracle.com/docs/cd/E11882_01/server.112/e10713/logical.htm



Transaktionskonzept ACID

- **A** wie **Atomicity**:
Transaktionen werden ganz oder gar nicht ausgeführt
- **C** wie **Consistency**:
Transaktionen überführen die Datenbank von einem konsistenten Zustand in einen anderen (konsistenten Zustand)
- **I** wie **Isolation**:
Transaktionen werden so ausgeführt, als wenn sie alleine auf der Datenbank operieren würden
- **D** wie **Durability**:
Transaktionen sind nach erfolgreichem Abschluss dauerhaft (*persistent*) und gehen auch durch Fehler nicht mehr verloren

Transaction Processing is data processing that actually works.



standardisierte Zugriffssprache: SQL

SQL: **S**tructured **Q**uery **L**anguage

- ist eine deklarative Sprache
- besteht aus Teilsprachen:
 - **DDL** - Data Definition Language / Data Description Language
... zur Umsetzung des konzeptionellen Entwurfes
 - **DML** - Data Manipulation Language
... ist die Datenverarbeitungssprache eines DBMS,
... Daten einfügen, ändern, löschen (und lesen) können
... schließt die Formulierung von Abfragen mit ein, **aber**
 - **DQL** - Data Query Language
... Sprachelemente zur Datenabfrage werden aufgrund ihrer Sonderstellung (manchmal)
einer eigenen Kategorie zugeordnet
 - **DCL** - Data Control Language
... Rechtevergabe
 - **TCL** - Transaction Control Language

SQL – DDL

Data Definition Language: DDL (am Beispiel Oracle 12c SQL)

```
CREATE TABLE person (  
  personid    NUMBER(5)      GENERATED ALWAYS AS IDENTITY NOT NULL,  
  vorname     VARCHAR2(20) NOT NULL,  
  nachname    VARCHAR2(20) NOT NULL,  
  alter       NUMBER(2)      NOT NULL,  
  stadt       VARCHAR2(50) DEFAULT 'Hamburg' NOT NULL,  
  strasse_nr  VARCHAR2(50) NOT NULL,  
  CONSTRAINT pk_person PRIMARY KEY (personid),  
  CONSTRAINT chk_alter CHECK (alter >= 18)  
);
```

SQL – DML

Data Manipulation Language: DML

```
INSERT INTO person
(personid, vorname, nachname, alter, stadt, strasse_nr)
VALUES
(DEFAULT, 'Max', 'Mustermann', 34, DEFAULT, 'Musterstr. 1');
```

```
INSERT INTO person
(personid, vorname, nachname, alter, stadt, strasse_nr)
VALUES
(DEFAULT, 'Alice', 'Supergirl', 18, 'London', 'Hopestreet 42');
```

```
COMMIT;
```



„Festschreiben“ der
Daten (TCL)

SQL – DQL

Data Query Language: DQL

```
SELECT nachname FROM person;
```

NACHNAME
Mustermann
Supergirl

```
SELECT nachname FROM person WHERE stadt = 'London';
```

NACHNAME
Supergirl

3 Relationen werden
miteinander „verknüpft“

```
SELECT person.nachname, artikel.preis  
FROM person, artikel, bestellung  
WHERE person.personid = bestellung.personid  
AND artikel.artikelnr = bestellung.artikelnr;
```

PERSON.NACHNAME	ARTIKEL.PREIS
Mustermann	3.99
Mustermann	129.65
Mustermann	7.88
Supergirl	13.42

Hersteller relationaler Datenbanksysteme & Produkte

ORACLE®



IBM®

Microsoft®

MySQL®

MariaDB

SYBASE®

PostgreSQL

Teradata
a division of  NCR

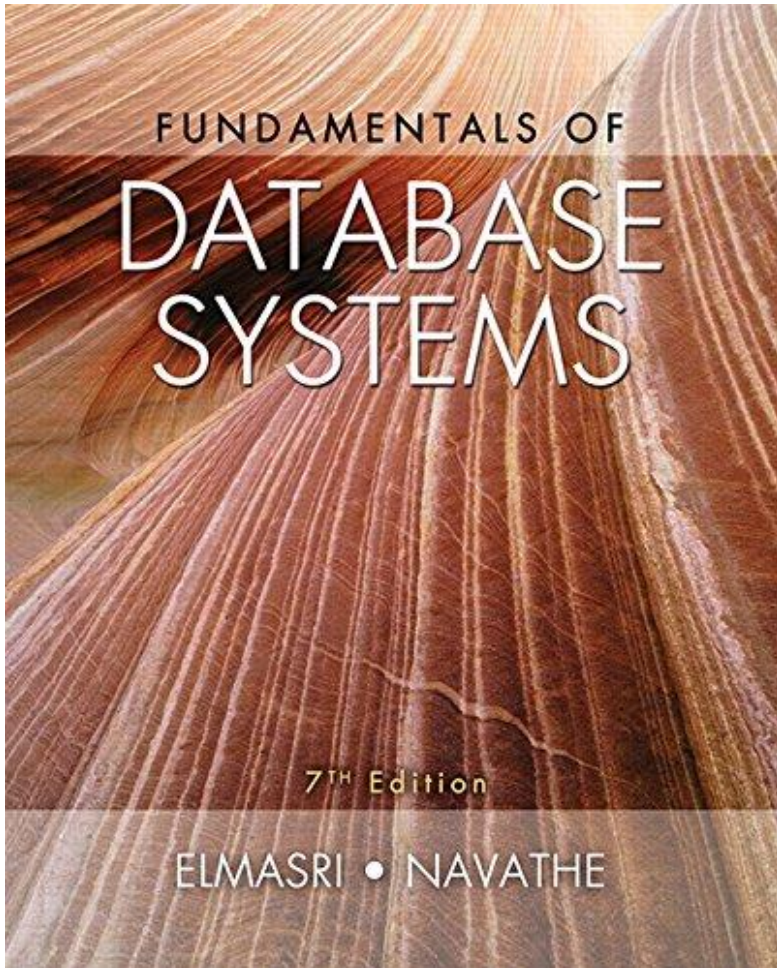
...

A word cloud of database names on a dark background. The names are in various sizes and colors (yellow, green, white). The names include:

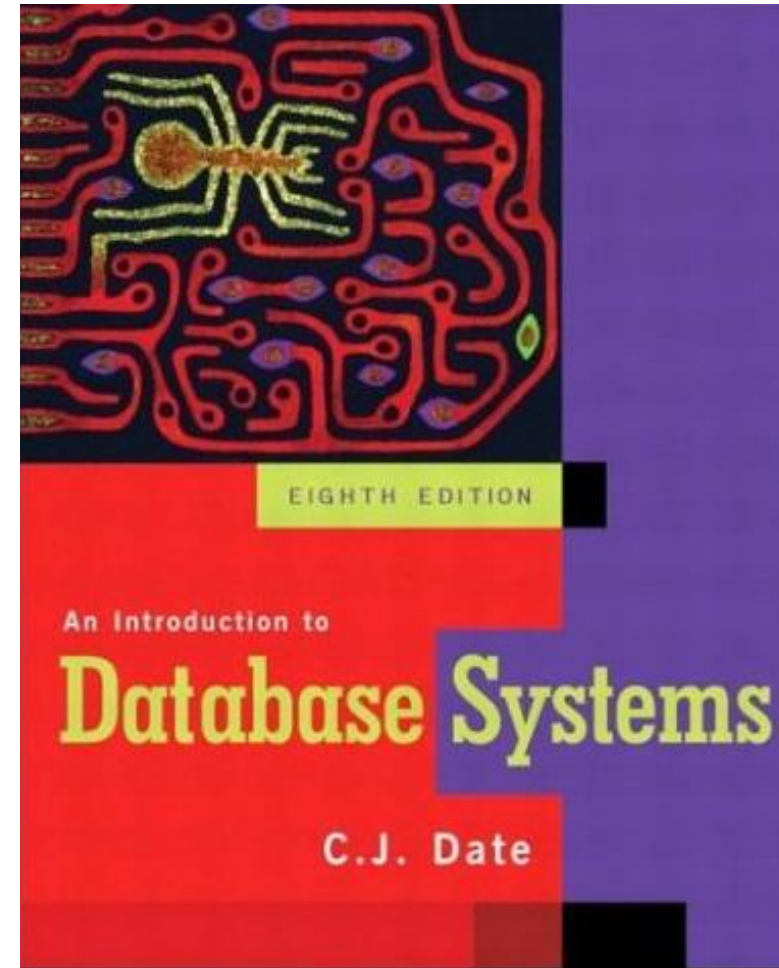
- ObjectStore
- Objectivitydb
- INGRES
- VERSANT
- SQLite
- TimesTen
- SystemR
- Redis
- MonetDB
- Informix
- Riak
- IMS
- Poet
- PostgreSQL
- DB2
- MariaDB
- HanaDB
- MongoDB
- Oracle
- MaxDB
- SQLServer
- MySQL
- CouchDB
- Cassandra
- Access
- FireBird
- GraphDB
- Infonyte
- CAP
- xindice
- Tamino
- NoSQL
- eXelon
- ADABAS
- eXist
- UDS
- dBASE
- ACID



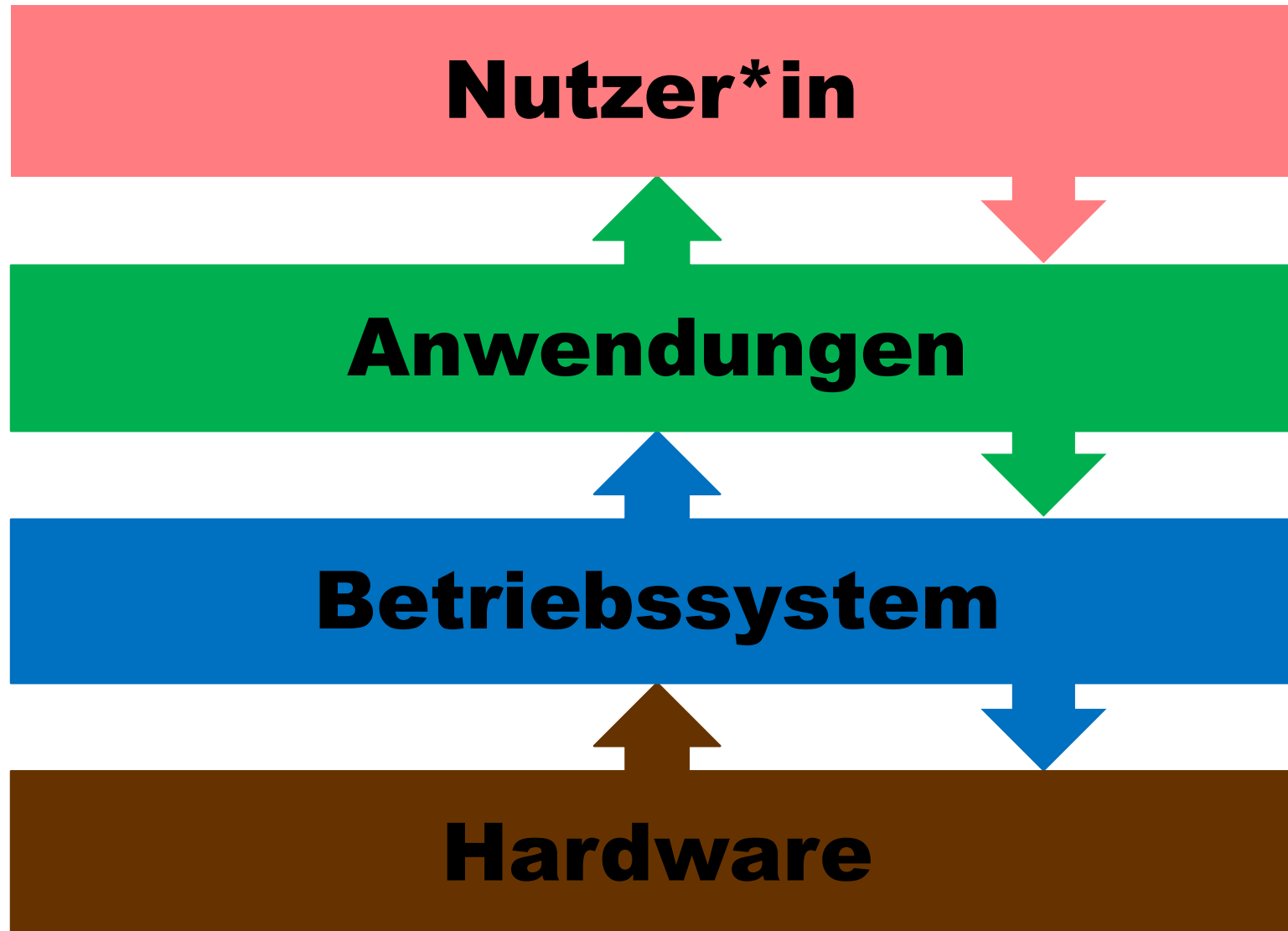
Datenbankliteratur



Ramez A. Elmasri, Shamkant B. Navathe:
Fundamentals of Database Systems.
7. Aufl., Addison-Wesley, 2015



C. J. Date:
An Introduction to Database Systems.
8. Aufl., Addison-Wesley, 2003



Lernziele Datenbanksysteme

- ... Vorteile von Datenbanksystemen gegenüber Dateisystemen erklären können
- ... Prinzip des relationalen Daten(bank)modells verstanden haben
- ... den Vorteil einer einheitlichen standardisierten Anfragesprache erklären können